UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA

ARQUITECTURA DE SOFTWARE · PRIMER SPRINT

Admisión UNSCH

Un proceso más claro, acompañado y digital

IA Sistema Integral de Gestión del Proceso de Admisión

DIEGO ALEJANDRO CORAS BENDAÑO Código: 27210102

Docente: Ing. Lizbeth Jaico Quispe · Código: 27210102

Arquitectura de Software · IS-488 | Ayacucho, 2026

Postular no debería implicar incertidumbre

Situaciones identificadas en el proceso actual de admisión



Una experiencia que necesita mejorar

El postulante puede necesitar atención presencial para corregir datos, aclarar pagos o completar su inscripción.



Correcciones

Trámites y desplazamientos.



Pagos

Dudas y errores costosos.



Seguimiento

Poca claridad del avance.



Orientación

Necesidad de ayuda oportuna.

Problemas preliminares: contrastar con encuestas y documentación institucional.

Una admisión digital, guiada y trazable

Mejorar la experiencia del postulante sin reemplazar los controles institucionales

Sistema Integral de Admisión

Portal web + agente de IA + seguimiento de trámites

01

Orientar

Chatbot contextual.

02

Inscribir

Datos y requisitos validados.

03

Pagar

Estado y alertas.

04

Seguir

Trámites y resultados.

Objetivo: prevenir errores y reducir gestiones presenciales evitables.

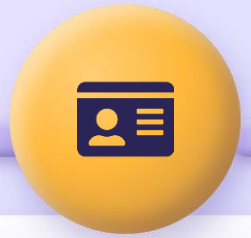
Del primer contacto a la postulación

Un recorrido asistido con información y verificaciones



Consultar

La IA explica modalidades.



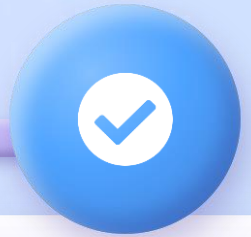
Registrar

DNI y datos verificados.



Pagar

Revisión previa y seguimiento.



Confirmar

Ficha, estado y alertas.



EJEMPLO Un postulante de Huanta completa su inscripción desde el celular y consulta el estado de su pago.

Requisitos que responden al problema

Funcionalidades prioritarias y una historia de usuario representativa

HISTORIA DE USUARIO · HU01

Como postulante, quiero recibir orientación y verificar mis datos antes de pagar, para evitar errores en mi inscripción.

RF01- 03

Registro, identidad y modalidad

RF04- 06

Inscripción, pagos y expedientes

RF07- 09

Exámenes, resultados y notificaciones

RF10 -12

Chatbot IA, seguimiento y auditoría

Identificadores y alcance propuestos; se consolidarán en el análisis de requisitos.

Qué condiciona la arquitectura

Drivers arquitectónicos propuestos

DA01

Alta demanda

Responder a picos de inscripciones.

DA02

Datos personales

Roles, cifrado y auditoría.



DA03

Pagos confiables

Evitar duplicidad y mostrar estados.

DA04

Integraciones

Aislar RENIEC y proveedores.

Las metas cuantitativas deberán definirse y validarse con pruebas.

Una API modular conecta todo el proceso

Arquitectura lógica simplificada



Despliegue propuesto: API Gateway + AWS Lambda; datos en RDS, caché y S3.

Monolito modular + Clean Architecture

Simplicidad operativa y separación de responsabilidades

ADR-001

Monolito modular

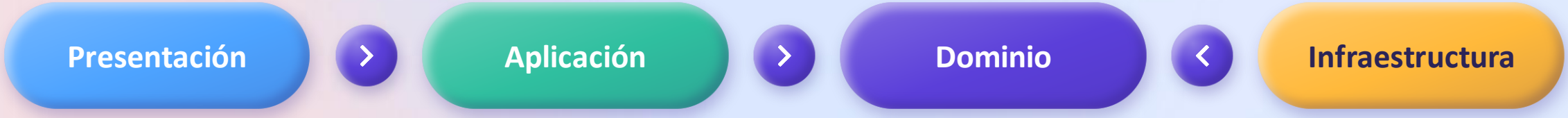
Una API desplegable; módulos por dominio.

ADR-002

Clean Architecture

Reglas del negocio aisladas de infraestructura.

REGLA DE DEPENDENCIA



Compromiso: los módulos se despliegan juntos y requieren límites internos claros.

Cuatro vistas de la misma solución

De los actores a las clases y contratos

1

Contexto

Postulantes · Personal

UNSCH ↔ RENIEC / Pagos

2

Contenedores

Portal · API · Datos

AWS + integraciones

3

Componentes

14 módulos funcionales

Casos de uso y adaptadores

4

Código

Clases e interfaces

Inscripción y pagos

Las vistas muestran la arquitectura propuesta, no un sistema ya desplegado.

Inscribir correctamente a un postulante

Flujo central del sistema y patrones de diseño

1
Validar
Identidad y requisitos

2
Registrar
Modalidad y datos

3
Pagar
Orden y confirmación

4
Notificar
Ficha y seguimiento

PATRONES DE DISEÑO

Repository
Persistencia

Adapter
RENIEC / pagos

Strategy
Reglas por modalidad

DIP
Contratos estables

Confirmación de pago únicamente mediante respuesta confiable del proveedor.

Una propuesta para postular con confianza

Primer sprint: análisis y arquitectura, no implementación productiva

Trabajo planteado

Problema · Requisitos · Drivers · Arquitectura · C4

Siguiente paso

Validar quejas · Prototipar IA · Probar pagos y usabilidad

Que postular a la UNSCH comience con orientación, no con incertidumbre.

Cómo se validará la calidad

Metas iniciales, sujetas a pruebas y ajuste

Rendimiento

p95 < 3 s

Carga con k6/JMeter

Disponibilidad

99.9 %

Objetivo · Monitoreo mensual

Seguridad

Por rol

Acceso según rol · Pruebas de autorización

Pagos

Sin duplicados

Idempotencia y conciliación

Usabilidad

Sin asistencia

Tareas sin asistencia · Pruebas con postulantes

Metas propuestas: no representan resultados medidos.

Módulos

- Módulo 01. Gestión de convocatorias y procesos de admisión
- Módulo 02. Registro de usuarios y validación de identidad
- Módulo 03. Inscripción de postulantes - Modalidad Ordinaria
- Módulo 04. Inscripción de postulantes - Modalidad Exonerados
- Módulo 05. Gestión de pagos y pasarela de pagos
- Módulo 06. Gestión y validación de expedientes
- Módulo 07. Programación y gestión de exámenes de admisión

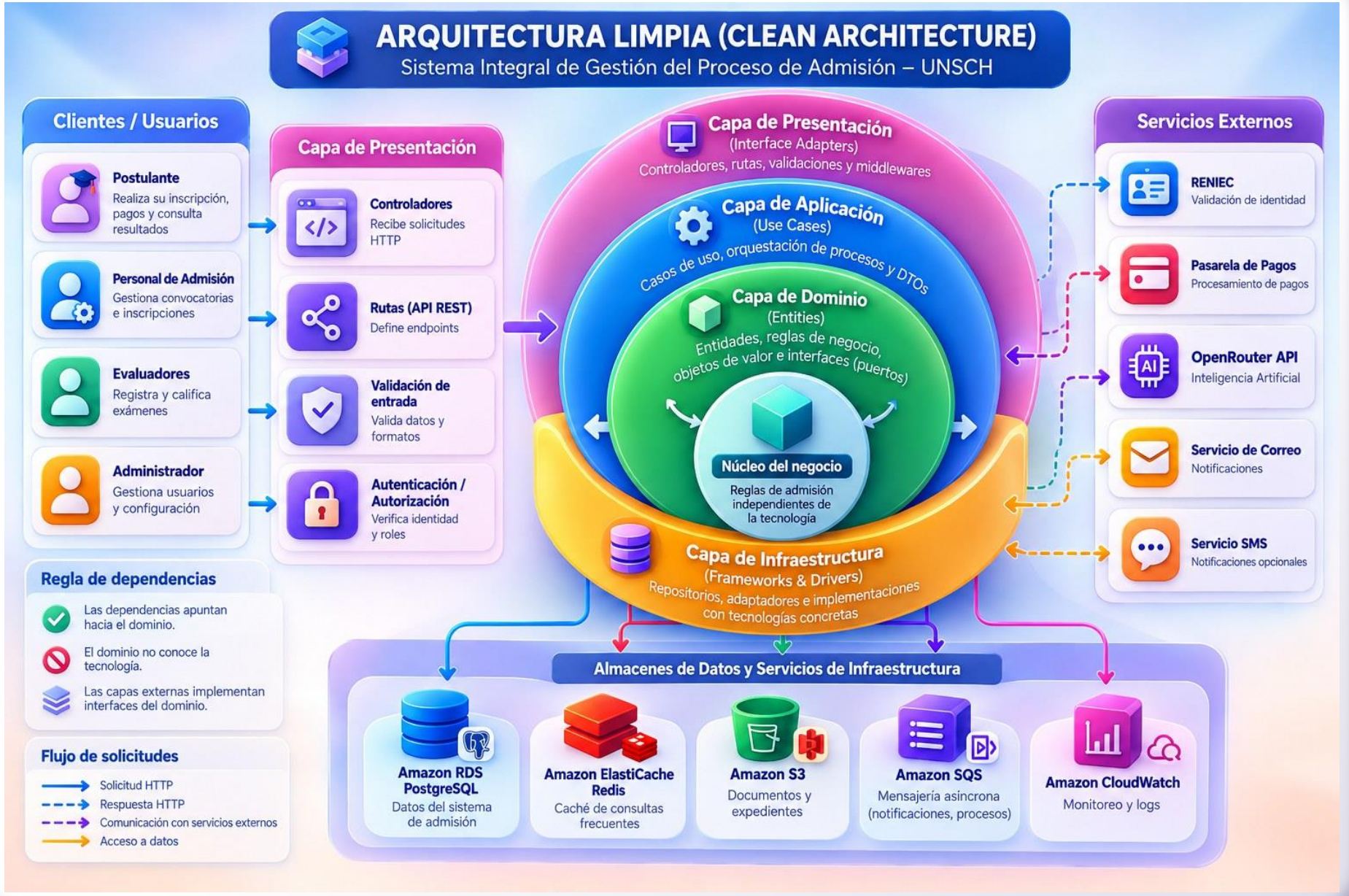
Módulos

- Módulo 08. Gestión de sedes, aulas y distribución de postulantes
- Módulo 09. Procesamiento, publicación y consulta de resultados
- Módulo 10. Chatbot inteligente de orientación y seguimiento - OpenRouter API
- Módulo 11. Notificaciones y comunicaciones
- Módulo 12. Reportes, estadísticas y panel de control
- Módulo 13. Administración, roles y seguridad
- Módulo 14. Configuración general e integraciones

Por qué estas decisiones

Respuestas breves para la sustentación

¿Por qué no microservicios?	El monolito modular simplifica el despliegue inicial.
¿Qué pasa si falla RENIEC o el pago?	Estado pendiente, error controlado y reintento seguro.
¿Qué debe confirmarse antes de implementar?	Convenios, reglas oficiales, proveedores y resultados de encuestas.


ARQUITECTURA LIMPIA (CLEAN ARCHITECTURE)
Sistema Integral de Gestión del Proceso de Admisión – UNSCH
Clientes / Usuarios
Postulante
Realiza su inscripción, pagos y consulta resultados
Personal de Admisión
Gestiona convocatorias e inscripciones
Evaluadores
Registra y califica exámenes
Administrador
Gestiona usuarios y configuración
Capa de Presentación
Controladores
Recibe solicitudes HTTP
Rutas (API REST)
Define endpoints
Validación de entrada
Valida datos y formatos
Autenticación / Autorización
Verifica identidad y roles
Capa de Presentación
(Interface Adapters)
Controladores, rutas, validaciones y middlewares
Capa de Aplicación
(Use Cases)
Casos de uso, orquestación de procesos y DTOs
Capa de Dominio
(Entities)
Entidades, reglas de negocio, objetos de valor e interfaces (puertos)
Núcleo del negocio
Reglas de admisión independientes de la tecnología
Capa de Infraestructura
(Frameworks & Drivers)
Repositorios, adaptadores e implementaciones con tecnologías concretas
Servicios Externos
RENIEC
Validación de identidad
Pasarela de Pagos
Procesamiento de pagos
OpenRouter API
Inteligencia Artificial
Servicio de Correo
Notificaciones
Servicio SMS
Notificaciones opcionales
Regla de dependencias
Las dependencias apuntan hacia el dominio.
El dominio no conoce la tecnología.
Las capas externas implementan interfaces del dominio.
Flujo de solicitudes
Solicitud HTTP
Respuesta HTTP
Comunicación con servicios externos
Acceso a datos
Almacenes de Datos y Servicios de Infraestructura
Amazon RDS PostgreSQL
Datos del sistema de admisión
Amazon ElastiCache Redis
Caché de consultas frecuentes
Amazon S3
Documentos y expedientes
Amazon SQS
Mensajería asíncrona (notificaciones, procesos)
Amazon CloudWatch
Monitoreo y logs


ARQUITECTURA MONOLÍTICA POR CAPAS
Sistema Integral de Gestión del Proceso de Admisión – UNSCH
Clientes / Usuarios
Postulante
Realiza su inscripción y consulta resultados
Personal de Admisión
Gestiona convocatorias e inscripciones
Evaluadores
Registra y califica exámenes
Administrador
Gestiona usuarios y configuración
HTTPS
Aplicación Web
SPA
(Navegador)
HTTPS
JSON
Backend – Monolito Modular (Node.js + Express)
Capa de Presentación
Controladores y rutas REST
Validación de entrada
Autenticación y autorización
Capa de Aplicación
Casos de uso
Orquestación de procesos
Flujos de negocio
Capa de Dominio
Entidades y objetos de valor
Reglas de negocio
Interfaces (puertos)
Capa de Infraestructura
Repositorios (PostgreSQL)
Adaptadores (servicios externos)
Implementaciones técnicas
HTTPS API
HTTPS REST
HTTPS REST
API Segura
HTTPS API
Sistemas Externos
RENIEC
Validación de identidad
Pasarela de Pagos
Procesamiento de pagos
OpenRouter API
Inteligencia Artificial
Servicio de Correo
Notificaciones
Servicio SMS
Notificaciones opcionales
Almacenes de Datos y Servicios de Infraestructura
Amazon RDS PostgreSQL
Datos del sistema de admisión
Amazon ElastiCache Redis
Caché de consultas frecuentes
Amazon S3
Documentos y expedientes
Amazon SQS
Mensajeria asíncrona (notificaciones, procesos)
Amazon CloudWatch
Monitoreo y logs

UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA

SOLID – MÓDULO DE PEDIDOS

Aplicación de principios SOLID en la gestión de inscripciones

Sistema Integral de Gestión del Proceso de Admisión – UNSCH



Capa de presentación (Controllers)

InscripcionController
- + crearInscripcion()
- + obtenerInscripcion()
- + listarInscripciones()

PagoController
- + generarOrdenPago()
- + confirmarPago()
- + consultarEstadoPago()

ExamenController
- + asignarExamen()
- + obtenerCronograma()
- + obtenerResultados()

Capa de aplicación (Servicios – Implementaciones)

InscripcionServiceImpl
- + crearInscripcion()
- + validarRequisitos()
- + obtenerInscripcion()

PagoServiceImpl
- + generarOrdenPago()
- + validarPago()
- + actualizarEstado()

ExamenServiceImpl
- + asignarExamen()
- + registrarResultado()
- + obtenerResultados()

Capa de dominio (Interfaces)

IInscripcionService
- + crearInscripcion()
- + validarRequisitos()
- + obtenerInscripcion()

IPagoService
- + generarOrdenPago()
- + validarPago()
- + actualizarEstado()

IExamenService
- + asignarExamen()
- + registrarResultado()
- + obtenerResultados()

Capa de infraestructura (Repositorios – Implementaciones)

InscripcionRepositoryImpl
- + guardar(inscripcion)
- + obtenerPorId(id)
- + listar()

PagoRepositoryImpl
- + guardar(pago)
- + obtenerPorId(id)
- + actualizarEstado()

ExamenRepositoryImpl
- + guardar(examen)
- + obtenerPorId(id)
- + listar()

Capa de datos (Base de datos)

BD UNSCH (Admisión)

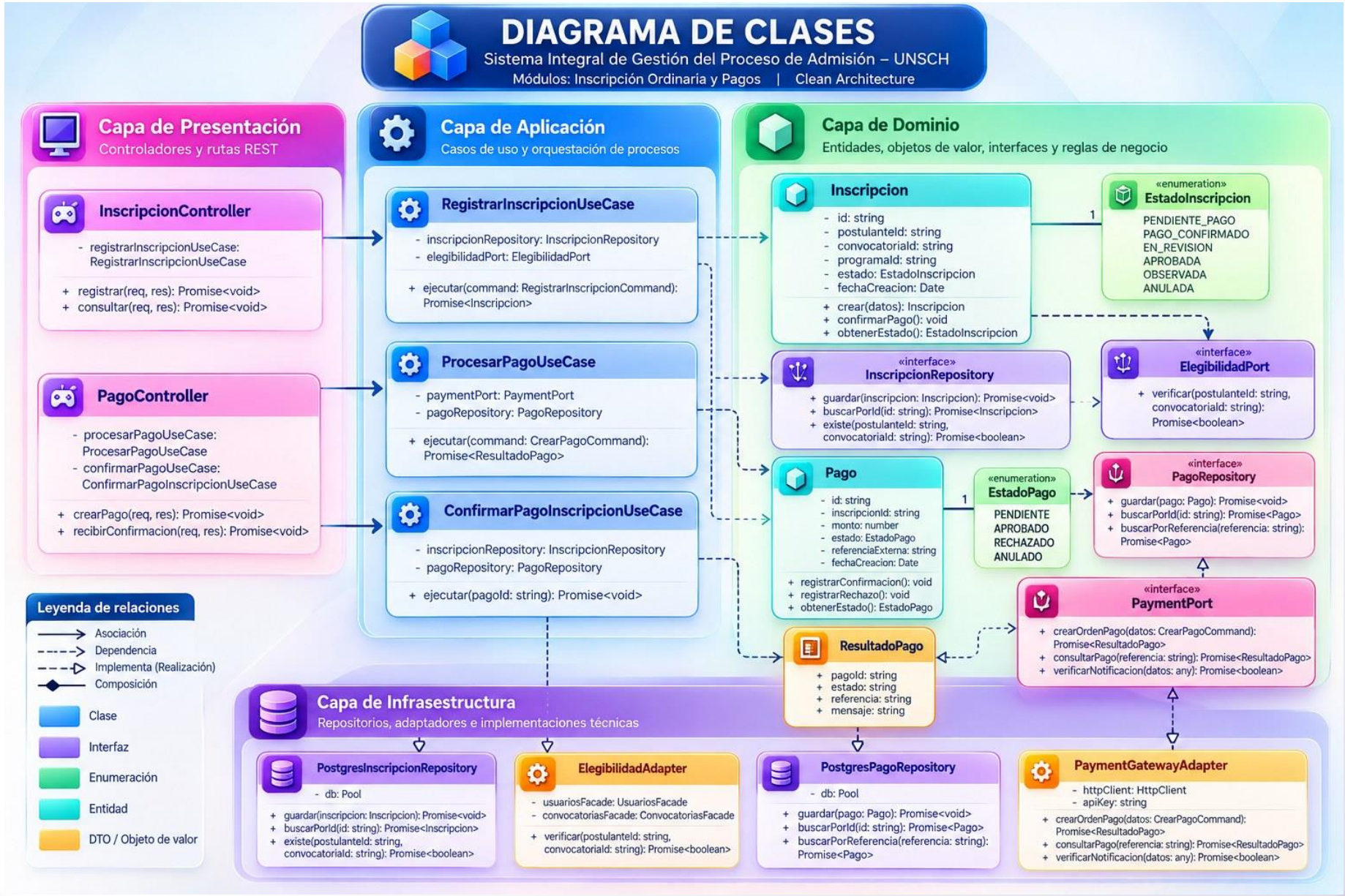
Principios SOLID aplicados

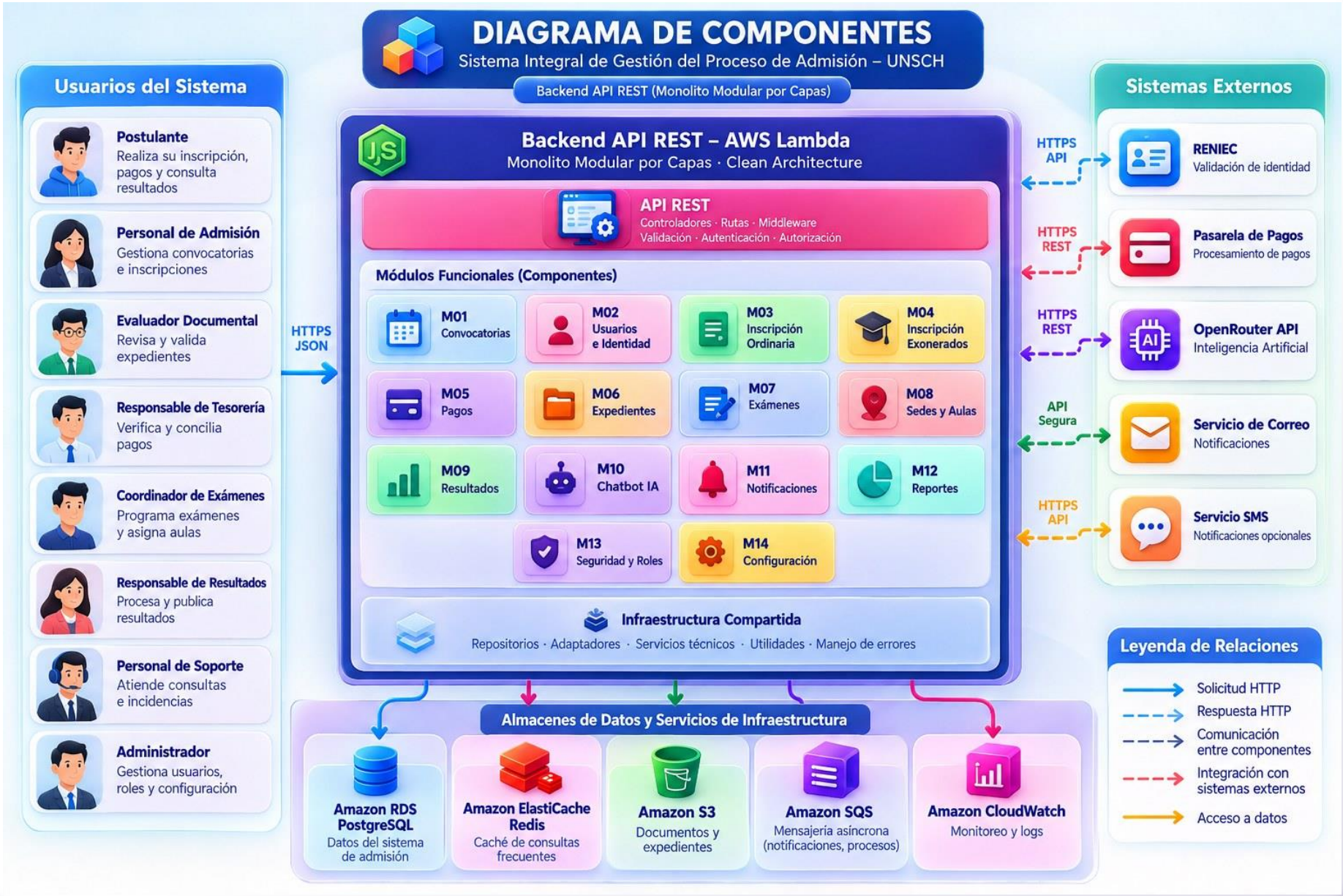
S	Single Responsibility	Cada clase tiene una única responsabilidad.
O	Open/Closed	Se pueden agregar nuevas funcionalidades sin modificar el código existente.
L	Liskov Substitution	Las implementaciones pueden sustituir a las interfaces.
I	Interface Segregation	Interfaces específicas por cada funcionalidad.
D	Dependency Inversion	Los módulos dependen de abstracciones (interfaces).

Leyenda

- Llamada / Uso
- Implementa (Realización)
- Depende de

- Controladores (Presentación)
- Servicios (Implementación)
- Interfaces (Dominio)
- Repositorios (Implementación)
- Base de datos (Datos)


DIAGRAMA DE CLASES
Sistema Integral de Gestión del Proceso de Admisión – UNSCH
Módulos: Inscripción Ordinaria y Pagos | Clean Architecture
Capa de Presentación
Controladores y rutas REST
InscripcionController
- registrarInscripcionUseCase: RegistrarInscripcionUseCase
+ registrar(req, res): Promise<void>
+ consultar(req, res): Promise<void>
PagoController
- procesarPagoUseCase: ProcesarPagoUseCase
- confirmarPagoUseCase: ConfirmarPagoInscripcionUseCase
+ crearPago(req, res): Promise<void>
+ recibirConfirmacion(req, res): Promise<void>
Capa de Aplicación
Casos de uso y orquestación de procesos
RegistrarInscripcionUseCase
- inscripcionRepository: InscripcionRepository
- elegibilidadPort: ElegibilidadPort
+ ejecutar(command: RegistrarInscripcionCommand): Promise<Inscripcion>
ProcesarPagoUseCase
- paymentPort: PaymentPort
- pagoRepository: PagoRepository
+ ejecutar(command: CrearPagoCommand): Promise<ResultadoPago>
ConfirmarPagoInscripcionUseCase
- inscripcionRepository: InscripcionRepository
- pagoRepository: PagoRepository
+ ejecutar(pagoId: string): Promise<void>
Capa de Dominio
Entidades, objetos de valor, interfaces y reglas de negocio
Inscripcion
- id: string
- postulanteId: string
- convocatoriaId: string
- programaId: string
- estado: EstadoInscripcion
- fechaCreacion: Date
+ crear(datos): Inscripcion
+ confirmarPago(): void
+ obtenerEstado(): EstadoInscripcion
1
«enumeration»
EstadoInscripcion
PENDIENTE_PAGO
PAGO_CONFIRMADO
EN_REVISION
APROBADA
OBSERVADA
ANULADA
«interface»
InscripcionRepository
+ guardar(inscripcion: Inscripcion): Promise<void>
+ buscarPorId(id: string): Promise<Inscripcion>
+ existe(postulanteId: string, convocatoriaId: string): Promise<boolean>
«interface»
ElegibilidadPort
+ verificar(postulanteId: string, convocatoriaId: string): Promise<boolean>
Pago
- id: string
- inscripcionId: string
- monto: number
- estado: EstadoPago
- referenciaExterna: string
- fechaCreacion: Date
+ registrarConfirmacion(): void
+ registrarRechazo(): void
+ obtenerEstado(): EstadoPago
1
«enumeration»
EstadoPago
PENDIENTE
APROBADO
RECHAZADO
ANULADO
«interface»
PagoRepository
+ guardar(pago: Pago): Promise<void>
+ buscarPorId(id: string): Promise<Pago>
+ buscarPorReferencia(referencia: string): Promise<Pago>
«interface»
PaymentPort
+ crearOrdenPago(datos: CrearPagoCommand): Promise<ResultadoPago>
+ consultarPago(referencia: string): Promise<ResultadoPago>
+ verificarNotificacion(datos: any): Promise<boolean>
ResultadoPago
+ pagoId: string
+ estado: string
+ referencia: string
+ mensaje: string
Leyenda de relaciones
Asociación
Dependencia
Implementa (Realización)
Composición
Clase
Interfaz
Enumeración
Entidad
DTO / Objeto de valor
Capa de Infrasestructura
Repositorios, adaptadores e implementaciones técnicas
PostgresInscripcionRepository
- db: Pool
+ guardar(inscripcion: Inscripcion): Promise<void>
+ buscarPorId(id: string): Promise<Inscripcion>
+ existe(postulanteId: string, convocatoriaId: string): Promise<boolean>
ElegibilidadAdapter
- usuariosFacade: UsuariosFacade
- convocatoriasFacade: ConvocatoriasFacade
+ verificar(postulanteId: string, convocatoriaId: string): Promise<boolean>
PostgresPagoRepository
- db: Pool
+ guardar(pago: Pago): Promise<void>
+ buscarPorId(id: string): Promise<Pago>
+ buscarPorReferencia(referencia: string): Promise<Pago>
PaymentGatewayAdapter
- httpClient: HttpClient
- apiKey: string
+ crearOrdenPago(datos: CrearPagoCommand): Promise<ResultadoPago>
+ consultarPago(referencia: string): Promise<ResultadoPago>
+ verificarNotificacion(datos: any): Promise<boolean>


DIAGRAMA DE COMPONENTES
Sistema Integral de Gestión del Proceso de Admisión – UNSCH
Backend API REST (Monolito Modular por Capas)
Usuarios del Sistema
Postulante
Realiza su inscripción, pagos y consulta resultados
Personal de Admisión
Gestiona convocatorias e inscripciones
Evaluador Documental
Revisa y valida expedientes
Responsable de Tesorería
Verifica y concilia pagos
Coordinador de Exámenes
Programa exámenes y asigna aulas
Responsable de Resultados
Procesa y publica resultados
Personal de Soporte
Atiende consultas e incidencias
Administrador
Gestiona usuarios, roles y configuración
HTTPS JSON
Backend API REST – AWS Lambda
Monolito Modular por Capas · Clean Architecture
API REST
Controladores · Rutas · Middleware
Validación · Autenticación · Autorización
Módulos Funcionales (Componentes)
M01 Convocatorias
M02 Usuarios e Identidad
M03 Inscripción Ordinaria
M04 Inscripción Exonerados
M05 Pagos
M06 Expedientes
M07 Exámenes
M08 Sedes y Aulas
M09 Resultados
M10 Chatbot IA
M11 Notificaciones
M12 Reportes
M13 Seguridad y Roles
M14 Configuración
Infraestructura Compartida
Repositorios · Adaptadores · Servicios técnicos · Utilidades · Manejo de errores
Sistemas Externos
HTTPS API
RENIEC
Validación de identidad
HTTPS REST
Pasarela de Pagos
Procesamiento de pagos
HTTPS REST
OpenRouter API
Inteligencia Artificial
API Segura
Servicio de Correo
Notificaciones
HTTPS API
Servicio SMS
Notificaciones opcionales
Almacenes de Datos y Servicios de Infraestructura
Amazon RDS PostgreSQL
Datos del sistema de admisión
Amazon ElastiCache Redis
Caché de consultas frecuentes
Amazon S3
Documentos y expedientes
Amazon SQS
Mensajería asíncrona (notificaciones, procesos)
Amazon CloudWatch
Monitoreo y logs
Leyenda de Relaciones
Solicitud HTTP
Respuesta HTTP
Comunicación entre componentes
Integración con sistemas externos
Acceso a datos

UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA

DIAGRAMA DE CONTENEDORES

Sistema Integral de Gestión del Proceso de Admisión – UNSCH



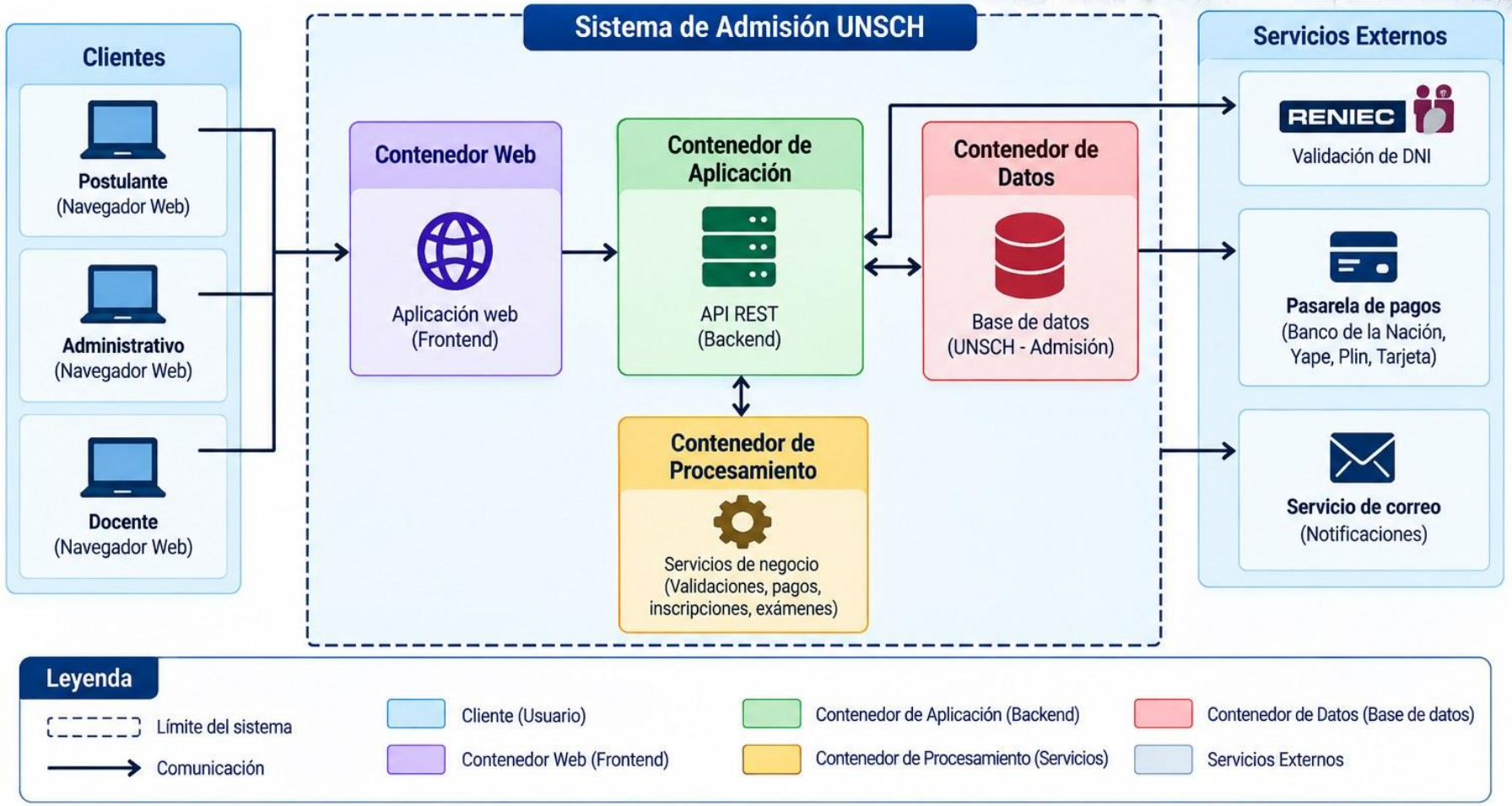



DIAGRAMA DE CONTENEDORES
Sistema Integral de Gestión del Proceso de Admisión – UNSCH
UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA
Sistema Integral de Gestión del Proceso de Admisión – UNSCH
Usuarios del Sistema
Postulante
Realiza inscripción, pagos y consulta resultados
Personal de Admisión
Gestiona convocatorias e inscripciones
Personal Administrativo
Gestiona usuarios, reportes y configuración
HTTPS (HTTPS/JSON)
Aplicación Web
Frontend
Interfaz web para postulantes y personal de la UNSCH
HTTPS (REST)
Amazon API Gateway
Gestiona las solicitudes y las redirige al backend
Invoca (Lambda)
Backend API REST
AWS Lambda
Monolito modular por capas (Clean Architecture)
Módulos principales:
M01 Convocatorias
M02 Usuarios e Identidad
M03 Inscripción Ordinaria
M04 Inscripción Exonerados
M05 Pagos
M06 Expedientes
M07 Exámenes
M08 Sedes y Aulas
M09 Resultados
M10 Chatbot IA
M11 Notificaciones
M12 Reportes
M13 Seguridad y Roles
M14 Configuración
Sistemas Externos
HTTPS API
RENIEC
Validación de identidad de postulantes
HTTPS REST
Pasarela de Pagos
Procesamiento de pagos de inscripción
HTTPS REST
OpenRouter API
Inteligencia Artificial para el chatbot
API Segura
Servicio de Correo
Envío de notificaciones a postulantes
HTTPS API
Servicio SMS (Twilio)
Envío de notificaciones opcionales
Almacenes de Datos y Servicios de Infraestructura (AWS)
Amazon RDS PostgreSQL
Datos del sistema de admisión
Amazon ElastiCache Redis
Caché de consultas frecuentes
Amazon S3
Documentos y expedientes
Amazon SQS
Mensajería asíncrona (notificaciones, procesos)
Leyenda
Persona (Usuario)
Contenedor (Frontend)
Contenedor (Backend)
Almacén de datos / Infraestructura
Sistema externo
Comunicación del usuario
Comunicación con sistema externo
Comunicación entre contenedores


DIAGRAMA DE CONTEXTO DEL SISTEMA
Sistema Integral de Gestión del Proceso de Admisión – UNSCH
UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA
Usuarios del Sistema
Postulante
Realiza su inscripción, paga y consulta resultados.
Personal de Admisión
Gestiona convocatorias e inscripciones.
Administrador
Gestiona usuarios, roles y configuración del sistema.
Evaluador Documental
Revisa y valida expedientes.
Coordinador de Exámenes
Programa exámenes y asigna aulas.
Responsable de Resultados
Procesa y publica resultados.
Personal de Soporte
Atiende consultas e incidencias.
Usa el sistema (inscripción y consulta)
Gestiona convocatorias e inscripciones
Administra usuarios y configuración
Valida expedientes
Organiza y ejecuta exámenes
Gestiona y publica resultados
Brinda soporte y atención
UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA
Sistema Integral de Gestión del Proceso de Admisión – UNSCH
Gestiona convocatorias, inscripciones, pagos, expedientes, exámenes, resultados y notificaciones del proceso de admisión.
Valida identidad (HTTPS / API)
Solicita y verifica pagos (HTTPS / REST)
Solicita respuestas (HTTPS / REST)
Envía notificaciones (HTTPS / API)
Envía avisos (HTTPS / API)
Sistemas Externos
RENIEC
Validación de identidad de postulantes.
Pasarela de Pagos
Procesamiento de pagos de inscripción.
OpenRouter API
Inteligencia Artificial para el chatbot.
Servicio de Correo
Envío de notificaciones a postulantes.
Servicio SMS (Twilio)
Envío de notificaciones opcionales.
UNSCH

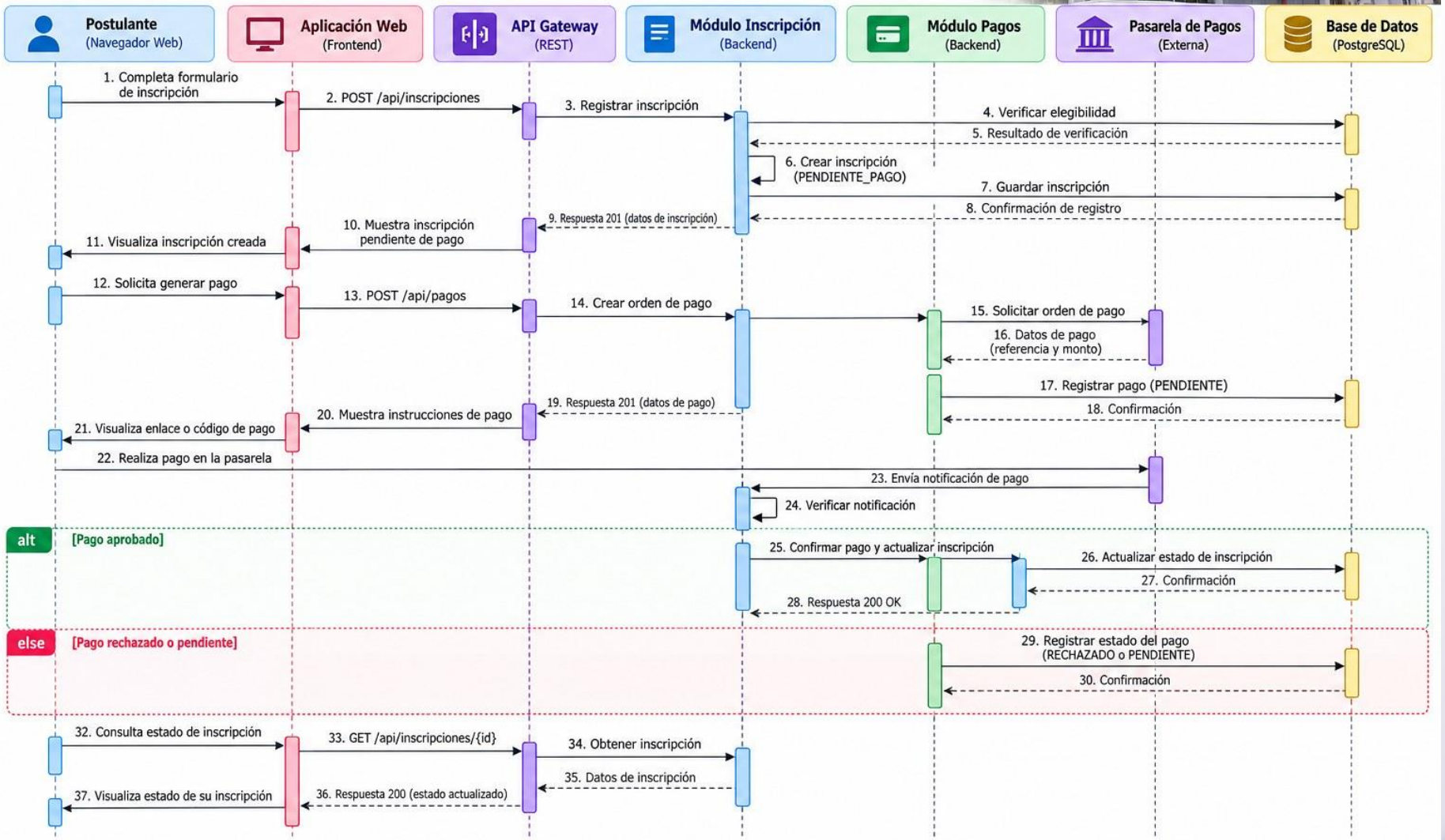
UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA

DIAGRAMA DE SECUENCIA

Registrar inscripción y confirmar pago

Sistema Integral de Gestión del Proceso de Admisión – UNSCH





UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA

PATRÓN ADAPTER

Integración con servicio externo (RENIEC)



Cliente
(Sistema de Admisión UNSCH)



Interfaz esperada por el cliente

ValidadorIdentidad

+ validarDNI(dni: String) : DatosPersona

Adaptador

ReniecAdapter

+ validarDNI(dni: String) : DatosPersona

- reniecApi: ReniecApi
- mapper: TransformadorDatos

Interfaz del servicio externo (RENIEC)

ReniecApi

+ consultarDNI(dni: String) : RespuestaReniec

Servicio Externo RENIEC



Flujo de interacción

1. El sistema de admisión solicita validar un DNI.



2. **ReniecAdapter** adapta la solicitud al formato de la API de RENIEC.



3. Se consume el servicio externo de RENIEC.



4. Se obtiene la respuesta y se transforma al formato esperado por el sistema.



UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA

PATRÓN FACADE

Simplificación del acceso a módulos del sistema

Sistema Integral de Gestión del Proceso de Admisión – UNSCH



UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA

PATRÓN OBSERVER

Notificación de cambios en el estado de inscripción

Sistema Integral de Gestión del Proceso de Admisión – UNSCH



Sujeto
(Inscripción)

Inscripcion

+ idInscripcion: int
+ estado: String
+ datosPostulante: String
+ observadores: List<Observador>

+ agregarObservador(o: Observador)
+ eliminarObservador(o: Observador)
+ notificarObservadores()
+ cambiarEstado(nuevoEstado: String)

Observadores
(Interesados en el cambio de estado)

NotificadorCorreo
+ actualizar(inscripcion: Inscripcion)
+ enviarCorreo()

NotificadorSMS
+ actualizar(inscripcion: Inscripcion)
+ enviarSMS()

NotificadorPortal
+ actualizar(inscripcion: Inscripcion)
+ mostrarNotificacion()

Evento
(Cambio de estado)

EstadoInscripcion

+ idInscripcion: int
+ estadoAnterior: String
+ estadoNuevo: String
+ fechaCambio: Date

se suscribe

se suscribe

se suscribe

genera evento

datos del evento

datos del evento

datos del evento

notifica cambio de estado

Leyenda

- Notificación (evento)
- Registro de observador (se suscribe)
- Datos del evento (estado actualizado)

UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA

PATRÓN FACTORY

Creación de objetos de inscripción según el tipo de modalidad

Sistema Integral de Gestión del Proceso de Admisión – UNSCH



Cliente
(Módulo de Inscripciones)

Modalidad de ingreso

Ordinaria

Crear inscripción

Solicita creación de inscripción (tipo de modalidad)

Retorna objeto de inscripción

InscripcionFactory
(Clase Creadora)

+ crearInscripcion(tipo: String) : Inscripcion

Según el tipo de modalidad

InscripcionOrdinaria
(Clase Concreta)

+ validarRequisitos()
+ calcularCosto()

InscripcionExtraordinaria
(Clase Concreta)

+ validarRequisitos()
+ calcularCosto()

InscripcionTraslado
(Clase Concreta)

+ validarRequisitos()
+ calcularCosto()

Inscripcion
(Producto Abstracto)

+ validarRequisitos()
+ calcularCosto()

Leyenda

- Solicitud / Llamada
- Respuesta
- Herencia / Implementación
- Clase Creadora (Factory)
- Producto Abstracto
- Producto Concreto (Modalidad Ordinaria)
- Producto Concreto (Modalidad Extraordinaria)
- Producto Concreto (Modalidad Traslado)
- Cliente

UNIVERSIDAD NACIONAL DE SAN CRISTÓBAL DE HUAMANGA

PATRÓN STRATEGY

Selección de método de pago en la inscripción

Sistema Integral de Gestión del Proceso de Admisión – UNSCH



Cliente
(Módulo de Pagos)

Método de pago
- Banco de la Nación
- Yape
- Plin
- Tarjeta

Pagar

Selecciona método de pago

Contexto
(Proceso de Pago)

ProcesadorPago
- estrategia: EstrategiaPago
+ setEstrategia(estrategia: EstrategiaPago)
+ procesarPago(monto: double): boolean

utiliza

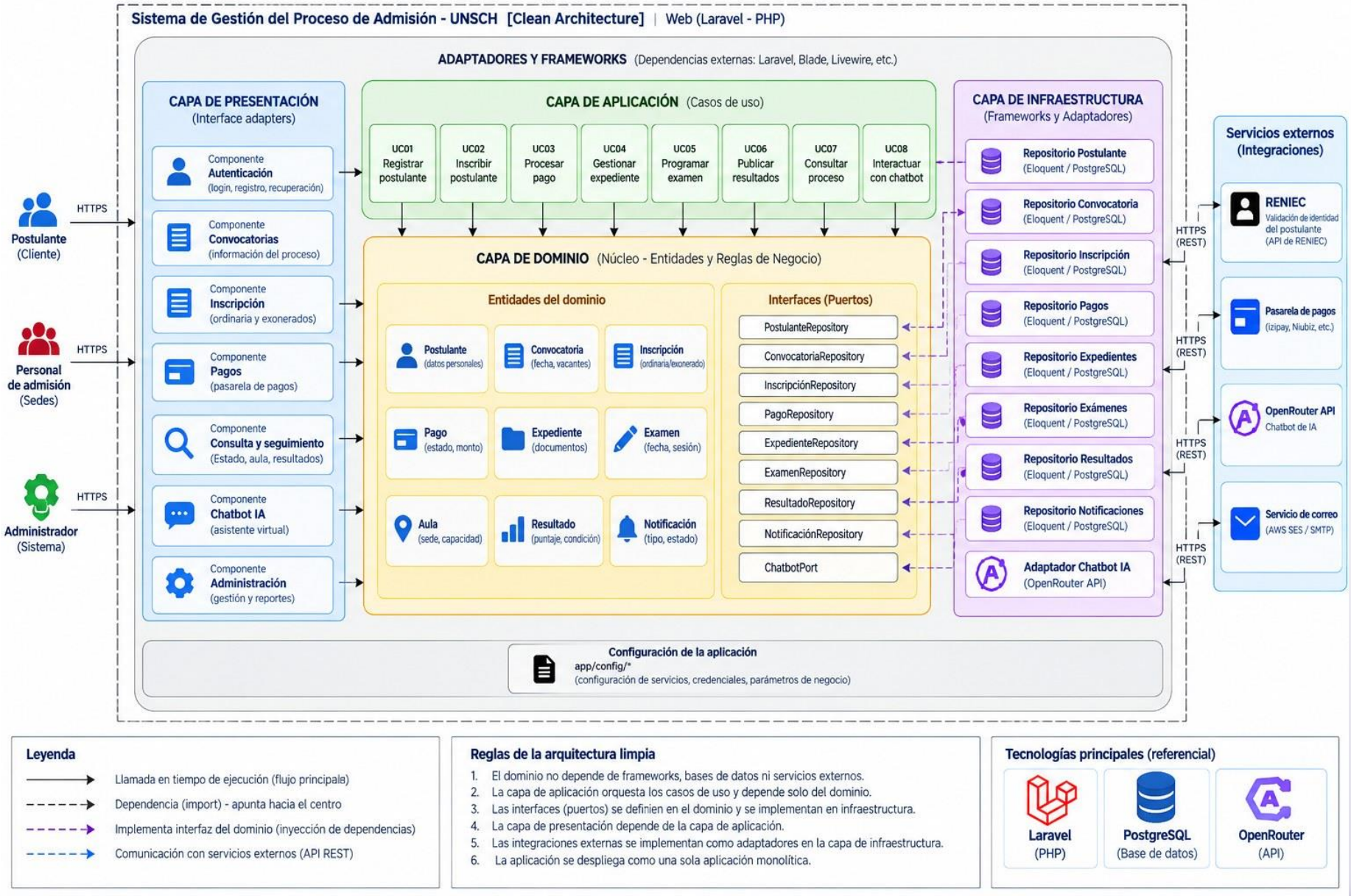
<< Interfaz >>
EstrategiaPago
+ pagar(monto: double): boolean

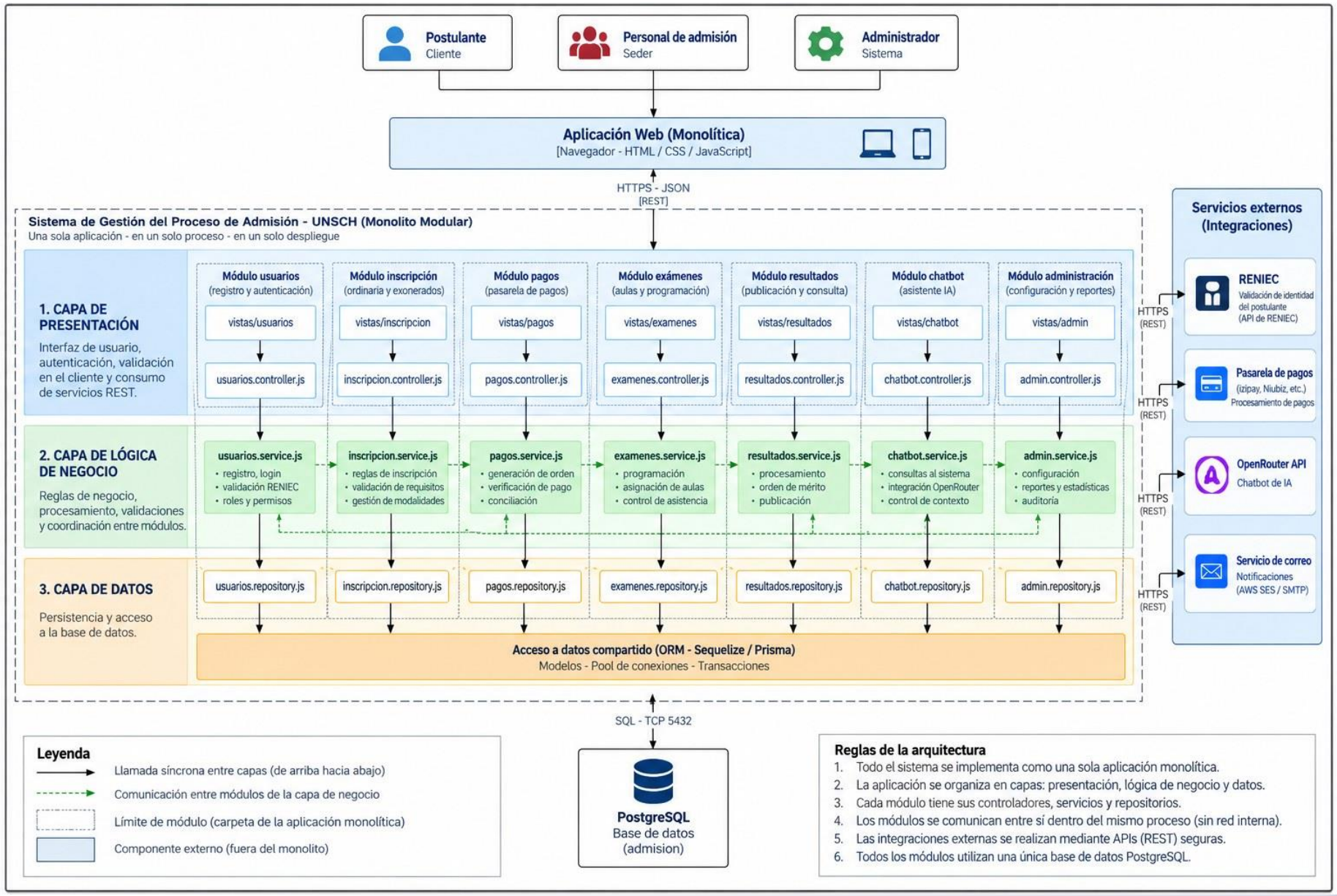
PagoBancoNacion
(Clase Concreta)
+ pagar(monto: double): boolean

PagoYape
(Clase Concreta)
+ pagar(monto: double): boolean

PagoPlin
(Clase Concreta)
+ pagar(monto: double): boolean

Leyenda
- Solicitud / Llamada
- Implementa
- Utiliza
- Cliente
- Contexto (Clase que usa la estrategia)
- Interfaz (Estrategia)
- Clases concretas (Estrategias)


Sistema de Gestión del Proceso de Admisión - UNSCH [Clean Architecture] | Web (Laravel - PHP)
ADAPTADORES Y FRAMEWORKS (Dependencias externas: Laravel, Blade, Livewire, etc.)
CAPA DE PRESENTACIÓN (Interface adapters)
Componente Autenticación (login, registro, recuperación)
Componente Convocatorias (información del proceso)
Componente Inscripción (ordinaria y exonerados)
Componente Pagos (pasarela de pagos)
Componente Consulta y seguimiento (Estado, aula, resultados)
Componente Chatbot IA (asistente virtual)
Componente Administración (gestión y reportes)
CAPA DE APLICACIÓN (Casos de uso)
UC01 Registrar postulante
UC02 Inscribir postulante
UC03 Procesar pago
UC04 Gestionar expediente
UC05 Programar examen
UC06 Publicar resultados
UC07 Consultar proceso
UC08 Interactuar con chatbot
CAPA DE DOMINIO (Núcleo - Entidades y Reglas de Negocio)
Entidades del dominio
Postulante (datos personales)
Convocatoria (fecha, vacantes)
Inscripción (ordinaria/exonerado)
Pago (estado, monto)
Expediente (documentos)
Examen (fecha, sesión)
Aula (sede, capacidad)
Resultado (puntaje, condición)
Notificación (tipo, estado)
Interfaces (Puertos)
PostulanteRepository
ConvocatoriaRepository
InscripciónRepository
PagoRepository
ExpedienteRepository
ExamenRepository
ResultadoRepository
NotificaciónRepository
ChatbotPort
CAPA DE INFRAESTRUCTURA (Frameworks y Adaptadores)
Repositorio Postulante (Eloquent / PostgreSQL)
Repositorio Convocatoria (Eloquent / PostgreSQL)
Repositorio Inscripción (Eloquent / PostgreSQL)
Repositorio Pagos (Eloquent / PostgreSQL)
Repositorio Expedientes (Eloquent / PostgreSQL)
Repositorio Exámenes (Eloquent / PostgreSQL)
Repositorio Resultados (Eloquent / PostgreSQL)
Repositorio Notificaciones (Eloquent / PostgreSQL)
Adaptador Chatbot IA (OpenRouter API)
Servicios externos (Integraciones)
RENIEC Validación de identidad del postulante (API de RENIEC)
Pasarela de pagos (izipay, Niubiz, etc.)
OpenRouter API Chatbot de IA
Servicio de correo (AWS SES / SMTP)
HTTPS (REST)
Postulante (Cliente)
Personal de admisión (Sedes)
Administrador (Sistema)
HTTPS
Configuración de la aplicación
app/config/*
(configuración de servicios, credenciales, parámetros de negocio)
Leyenda
Llamada en tiempo de ejecución (flujo principal)
Dependencia (import) - apunta hacia el centro
Implementa interfaz del dominio (inyección de dependencias)
Comunicación con servicios externos (API REST)
Reglas de la arquitectura limpia
1. El dominio no depende de frameworks, bases de datos ni servicios externos.
2. La capa de aplicación orquesta los casos de uso y depende solo del dominio.
3. Las interfaces (puertos) se definen en el dominio y se implementan en infraestructura.
4. La capa de presentación depende de la capa de aplicación.
5. Las integraciones externas se implementan como adaptadores en la capa de infraestructura.
6. La aplicación se despliega como una sola aplicación monolítica.
Tecnologías principales (referencial)
Laravel (PHP)
PostgreSQL (Base de datos)
OpenRouter (API)


Postulante
Cliente
Personal de admisión
Seder
Administrador
Sistema
Aplicación Web (Monolítica)
[Navegador - HTML / CSS / JavaScript]
HTTPS - JSON
[REST]
Sistema de Gestión del Proceso de Admisión - UNSCH (Monolito Modular)
Una sola aplicación - en un solo proceso - en un solo despliegue
1. CAPA DE PRESENTACIÓN
Interfaz de usuario, autenticación, validación en el cliente y consumo de servicios REST.
2. CAPA DE LÓGICA DE NEGOCIO
Reglas de negocio, procesamiento, validaciones y coordinación entre módulos.
3. CAPA DE DATOS
Persistencia y acceso a la base de datos.
Módulo usuarios
(registro y autenticación)
Módulo inscripción
(ordinaria y exonerados)
Módulo pagos
(pasarela de pagos)
Módulo exámenes
(aulas y programación)
Módulo resultados
(publicación y consulta)
Módulo chatbot
(asistente IA)
Módulo administración
(configuración y reportes)
vistas/usuarios
vistas/inscripcion
vistas/pagos
vistas/examenes
vistas/resultados
vistas/chatbot
vistas/admin
usuarios.controller.js
inscripcion.controller.js
pagos.controller.js
examenes.controller.js
resultados.controller.js
chatbot.controller.js
admin.controller.js
usuarios.service.js
• registro, login
• validación RENIEC
• roles y permisos
inscripcion.service.js
• reglas de inscripción
• validación de requisitos
• gestión de modalidades
pagos.service.js
• generación de orden
• verificación de pago
• conciliación
examenes.service.js
• programación
• asignación de aulas
• control de asistencia
resultados.service.js
• procesamiento
• orden de mérito
• publicación
chatbot.service.js
• consultas al sistema
• integración OpenRouter
• control de contexto
admin.service.js
• configuración
• reportes y estadísticas
• auditoría
usuarios.repository.js
inscripcion.repository.js
pagos.repository.js
examenes.repository.js
resultados.repository.js
chatbot.repository.js
admin.repository.js
Acceso a datos compartido (ORM - Sequelize / Prisma)
Modelos - Pool de conexiones - Transacciones
Servicios externos (Integraciones)
HTTPS (REST)
RENIEC
Validación de identidad del postulante (API de RENIEC)
Pasarela de pagos
(izipay, Niubiz, etc.)
Procesamiento de pagos
OpenRouter API
Chatbot de IA
Servicio de correo
Notificaciones
(AWS SES / SMTP)
SQL - TCP 5432
PostgreSQL
Base de datos
(admision)
Leyenda
Llamada síncrona entre capas (de arriba hacia abajo)
Comunicación entre módulos de la capa de negocio
Límite de módulo (carpeta de la aplicación monolítica)
Componente externo (fuera del monolito)
Reglas de la arquitectura
1. Todo el sistema se implementa como una sola aplicación monolítica.
2. La aplicación se organiza en capas: presentación, lógica de negocio y datos.
3. Cada módulo tiene sus controladores, servicios y repositorios.
4. Los módulos se comunican entre sí dentro del mismo proceso (sin red interna).
5. Las integraciones externas se realizan mediante APIs (REST) seguras.
6. Todos los módulos utilizan una única base de datos PostgreSQL.